

RAG-Based Chatbot System - Complete Project Requirements & Implementation Plan

This document describes the complete requirements, architecture, implementation flow, backend APIs, database design, and RAG implementation approach for a Retrieval-Augmented Generation (RAG) based chatbot system using Python FastAPI.

1. Project Overview

Build an AI chatbot system where:

User Portal: Users ask questions to the chatbot.

Admin Portal: Admin uploads PDF documents.

The system uses RAG (Retrieval-Augmented Generation) to retrieve relevant document information and generate accurate AI responses.

2. Functional Requirements

User Features:

- Ask questions to chatbot
- View chatbot answers
- Chat history
- Streaming response support

Admin Features:

- Upload PDF files
 - View uploaded documents
 - Delete/reprocess documents
 - Monitor embeddings status
-

3. System Architecture

Frontend:

- React / Next.js
- Tailwind CSS
- Chat UI

Backend:

- Python FastAPI
- REST APIs
- Async processing

AI Layer:

- LangChain or LlamaIndex

- OpenAI / Azure OpenAI
- Embedding model
- Vector database

Storage:

- PostgreSQL
 - Vector DB (FAISS / ChromaDB / PGVector)
 - File Storage
-

4. RAG Flow

Document Upload Flow:

1. Admin uploads PDF
2. Backend extracts text
3. Text is split into chunks
4. Embeddings generated
5. Stored in vector database

Question Answer Flow:

1. User asks question
 2. Question embedding created
 3. Similar chunks retrieved
 4. Context sent to LLM
 5. AI generates final answer
 6. Response displayed to user
-

5. Backend API Design

Admin APIs:

- POST /upload-pdf
- GET /documents
- DELETE /documents/{id}

Chat APIs:

- POST /chat
- GET /chat-history

Health APIs:

- GET /health
-

6. Database Tables

documents

- id
- filename
- uploaded_by
- upload_date

- processing_status

chat_history

- id
 - question
 - answer
 - timestamp
-

7. Chunking Strategy

Recommended Chunk Size: 500–1000 tokens

Overlap: 100–200 tokens

This improves semantic retrieval and avoids missing context.

8. Suggested Folder Structure

```
backend/  
■■■ app/  
■■■ api/  
■■■ services/  
■■■ rag/  
■■■ models/  
■■■ database/  
■■■ utils/  
■■■ main.py  
■■■ requirements.txt
```

9. Future Enhancements

- Multi-document search
 - OCR support for scanned PDFs
 - Voice chatbot
 - Multi-language support
 - Analytics dashboard
 - Feedback learning
 - Hybrid search (keyword + vector)
-

10. Recommended RAG Workflow

Best Recommended Flow:

Admin Upload PDF → Extract Text → Chunking → Generate Embeddings → Store in Vector DB → User Question → Generate Query Embedding → Similarity Search → Retrieve Context → Send Context + Question to LLM →

Generate Final Response
